Juego de reflejos sobre Pochoco SoC

Proyecto 2 | Arquitectura de Computadores

Universidad de los Andes · Semestre 2026-2
Borrador para revisión
Sebastián Germain · Valentina Pizarro · Florencia Osandón

1. Objetivo y arquitectura

El sistema implementa un juego de reflejos en assembly para el Espino Core sobre la Go Board. El módulo game_top instancia pochoco_soc y selecciona sw/game.hex. El procesador ejecuta el programa desde una RAM de 512 palabras de 32 bits. Los periféricos se controlan mediante direcciones de memoria. El juego utiliza los registros x0 a x15 y evita multiplicaciones, divisiones y desplazamientos de la ALU.

2. Modificaciones al RTL

En pochoco_periph se incorporó un contador libre de 32 bits que aumenta una vez por ciclo. Un registro led_start captura su valor en el mismo flanco que acepta una escritura al registro de LED. Por tanto, al escribir el LED objetivo se establece también la referencia temporal de esa ronda. Los botones pasan por dos registros consecutivos para sincronizarlos con el reloj. Esto reduce el riesgo de metastabilidad, pero no elimina por sí solo el rebote mecánico.

Mapa de periféricos

0x80000000: escritura de los dos dígitos, un nibble por dígito.
0x80000004: escritura de los cuatro LED y captura de led_start.
0x80000008: lectura de botones sincronizados, un bit por botón.
0x8000000C: lectura del contador libre.
0x80000010: lectura de led_start.

Inicialización

El contador por_cnt de pochoco_soc se amplió de 4 a 16 bits. Mantiene el reset activo hasta alcanzar 65535, aproximadamente 2,62 ms a 25 MHz. Después de este cambio se reportaron tres encendidos consecutivos correctos. Es evidencia de mejora del arranque en la placa probada; no demuestra por sí sola la causa física del fallo anterior. No se modificó el RTL del procesador Espino respecto de la base recibida.

Temporización y funcionamiento

3. Secuencia del juego

Al iniciar se muestra 00 y se espera una pulsación seguida de liberación. Cada ronda comienza con los cuatro LED encendidos durante al menos tres segundos. Luego se selecciona un único LED, se captura el inicio y se consultan los botones. Una respuesta correcta suma su tiempo y aumenta el número de aciertos. El resultado individual permanece unos dos segundos. Tras diez aciertos se muestra el promedio y los cuatro LED quedan encendidos.

Error y liberación de botones

Una pulsación distinta del objetivo, incluyendo múltiples botones, muestra EE durante aproximadamente un segundo y vuelve a preparar la ronda. Se conservan la suma y el número de aciertos. Se añadió un límite de respuesta de diez segundos con el mismo comportamiento de error. La rutina release exige 20 ms continuos sin botones antes de continuar; el flanco inicial de pulsación se acepta por sondeo, sin filtro adicional de 20 ms.

4. Conversión del tiempo

Frecuencia: 25000000 ciclos/s. Un ciclo dura 40 ns.
Tres segundos = 75000000 ciclos = 18311 × 4096 - 1856.
Una décima de segundo = 2500000 ciclos = 610 × 4096 + 1440.
Liberación de botones: 500000 ciclos = 20 ms.

Medición y promedio

Para cada acierto se calcula C = (contador_actual - led_start) módulo 2^32. La captura de inicio coincide con el cambio de LED. El final se registra mediante una lectura del contador posterior a detectar la pulsación, por lo que incluye una pequeña latencia de sincronización y sondeo. El tiempo mostrado es floor(C / 2500000). La división se implementa por restas sucesivas.
Se acumulan los ciclos sin truncar cada respuesta: S = C1 + ... + CN. El promedio mostrado es floor(S / (N × 2500000)), con N = 10 en la entrega. El divisor se obtiene sumando 2500000 N veces. Con diez respuestas menores de diez segundos, S permanece por debajo de 2500000000 y cabe en 32 bits sin signo.

Representación decimal

La rutina show_decimal obtiene decenas y unidades mediante restas de diez y escribe decenas × 16 + unidades. Así, el display 14 significa 14 décimas, es decir 1,4 s. La resta modular tolera un desbordamiento del contador libre, que ocurre aproximadamente cada 171,8 s; los intervalos medidos son mucho menores.

Selección del LED y assembler

5. Generación pseudoaleatoria

La semilla se toma del contador cuando el jugador inicia la partida. En cada ronda se calcula estado = 5 × estado + contador + 1, con aritmética de 32 bits. La multiplicación por cinco se realiza con sumas. Después se conservan once bits con AND 2047 y se obtiene el resto módulo 251 mediante restas. Los dos bits inferiores eligen un índice entre 0 y 3. A partir de 1, se duplica el valor tantas veces como indique el índice para producir la máscara 1, 2, 4 u 8.
El instante humano y los tiempos de las rondas influyen en la secuencia. Se permiten repeticiones de un LED. El método no garantiza distribución uniforme ni tiene fines criptográficos; su variación perceptible debe observarse en varias partidas.

6. Assembler propio

assembler.py usa únicamente la biblioteca estándar de Python. La primera pasada elimina comentarios, reconoce etiquetas y constantes y asigna direcciones en pasos de cuatro bytes. La segunda pasada codifica las instrucciones. Las etiquetas de salto se convierten en desplazamientos relativos al PC de la instrucción. Cada palabra se escribe con ocho dígitos hexadecimales en una línea del archivo de salida.

Sintaxis y validación

Se aceptan registros x0 a x15, operandos separados por comas, accesos offset(xN), etiquetas terminadas en dos puntos y comentarios #, // o punto y coma. Se admiten .text, .section .text, .global/.globl y .equ con un número o una constante anterior. No se aceptan expresiones generales.
Los inmediatos I/S deben caber en 12 bits con signo; los saltos B/J en 13/21 bits con signo y alineados a cuatro bytes. LUI/AUIPC reciben el campo de 20 bits sin signo. Se rechazan registros fuera de rango, instrucciones desconocidas, símbolos duplicados o ausentes y cantidades incorrectas de operandos.

Instrucciones del juego

lui, addi, add, sub, andi, lw, sw, beq, bne, bltu, bgeu, jal y jalr. El assembler también codifica otras operaciones del subconjunto descrito en su código, pero no genera instrucciones de desplazamiento, multiplicación ni división.

Ejemplo de codificación

addi x1, x0, 15 → 00f00093.
Inmediato = 0x00F; rs1 = 0; funct3 = 0; rd = 1; opcode = 0x13.
Palabra = (15 << 20) | (0 << 15) | (0 << 12) | (1 << 7) | 0x13.
Otro ejemplo: sw x1, 4(x2) → 00112223. Un salto a su propia etiqueta, jal x0, fin, con fin en esa misma instrucción, produce 0000006f.

Verificación y reproducción

7. Verificaciones sobre los archivos

Se regeneró en memoria la imagen de game.s y se comparó con game.hex del ZIP funcional: coincidencia de 111 instrucciones, equivalentes a 444 bytes. Se comprobaron cinco codificaciones conocidas y el rechazo de registro x16, inmediato 2048 en addi, etiqueta inexistente, etiqueta duplicada e instrucción slli. El resultado está en docs/pruebas_assembler.txt. Estas verificaciones no constituyen una simulación RTL ni sustituyen las pruebas físicas.

Pruebas físicas reportadas

Durante el trabajo se confirmó la correspondencia entre botones y LED, la lectura 00 sin botón y 01 con SW1, y una prueba de contador con espera de un segundo y retorno de subrutina. Se completó una partida con resultado promedio 14 y se observó EE seguido de repetición de ronda ante un botón incorrecto.
Con el reset ampliado y marcas de diagnóstico se reportaron tres encendidos consecutivos correctos. Luego se retiraron las marcas y se confirmó el inicio desde 00. Estas observaciones fueron comunicadas por el operador; este borrador no adjunta fotografías ni una tabla de tiempos medidos externamente.

8. Compilación y programación

En la terminal de OSS CAD Suite y desde la raíz del proyecto:
python3 assembler.py sw/game.s sw/game.hex
python3 build_game.py --program
El segundo comando ya incluye el primero. Sintetiza game_top, usa nextpnr-ice40 con HX1K, encapsulado vq100 y objetivo de 25 MHz, genera game_top.bin y programa mediante iceprog. El README contiene los pasos completos. El Makefile de sw pertenece a la base y no se utiliza porque invoca un assembler externo.

Demostración y revisión pendiente

Para demostrar el cambio de una constante, puede usarse RONDAS = 3, regenerar y programar; después debe restaurarse a 10. Antes de entregar se debe repetir la partida completa con esta versión final, verificar expresamente la conservación de aciertos tras un error intermedio, comprobar el timeout y observar la variación de LED entre partidas. Añadir evidencias y publicar el repositorio. No se atribuyen cifras de utilización o frecuencia máxima de compilaciones anteriores a esta versión final.

Conclusión

El proyecto integra el control por assembly, el acceso a periféricos y la medición mediante un contador de hardware. La captura del inicio junto a la escritura de LED permite definir una referencia temporal común. El assembler propio genera la imagen ejecutada en memoria. Las pruebas reportadas muestran funcionamiento del juego y una mejora del arranque al ampliar el reset; quedan las verificaciones finales y evidencias indicadas antes de cerrar la entrega.

Referencias y asistencia

Enunciado de Proyecto 2, Arquitectura de Computadores, Universidad de los Andes, 2026-2.
Código base de Nicolás Villegas: github.com/nic0villegasc/pochoco_soc (ZIP proporcionado).
Versión funcional y pruebas comunicadas durante el desarrollo. Se utilizó ChatGPT como apoyo para código, diagnóstico y documentación. Revisar las reglas del curso y validar personalmente el contenido antes de entregar.